

Module 03 — Preprocessing & Feature Engineering

src/preprocessing.py • 279 lines

In one sentence

Turns the raw interaction log into the tables and matrices the models need.

1. What this module does

Models cannot read a list of events. They need a **matrix**: rows for customers, columns for products, and a number in each cell. This module does that conversion, and builds several supporting feature tables at the same time.

It produces seven artifacts. The three matrices are the main ones - explicit ratings, mean-centred ratings, and binary cart/purchase intent. The other four are feature tables describing products and customers, which are what make cold-start recommendations possible.

2. Concepts covered

Every idea this module uses, in plain terms.

Concept	In simple terms	Where it is used here
User-item matrix	A grid of who rated what	<code>create_user_item_matrix()</code>
Pivot table	Turning a long list into a wide grid	<code>pandas pivot_table</code>
Mean centring	Subtract each person's average rating	<code>create_normalized_user_item_matrix()</code>
Implicit feedback	Actions, not stars	<code>create_implicit_feedback_matrix()</code>
One-hot encoding	Turn a category into 0/1 columns	<code>pd.get_dummies</code> on category, brand, segment
Min-max scaling	Squash numbers into the 0-1 range	Price and age scaling
float32 vs float64	Half the memory, same accuracy here	All matrices stored as float32
Reindexing	Keep rows that have no data	Preserving zero-interaction products
Vectorisation	Operate on whole columns, not row by row	<code>groupby().transform()</code> instead of <code>.apply()</code>

3. How it works, step by step

- 1. Explicit rating matrix.** Take only the rows that have a star rating and pivot them into a customer x product grid. Empty cells become 0.
- 2. Mean-centred matrix.** Subtract each customer's own average rating from their scores, so a generous rater and a harsh rater become comparable.
- 3. Implicit matrix.** Mark 1 wherever a customer added to cart or bought, 0 otherwise. This works even for the 31% of rows with no star rating.
- 4. Product popularity features.** Count interactions, purchases and revenue per product, work out the conversion rate, and flag the long tail.

- 5. Product content features.** Scale price and quality to 0-1, one-hot encode category, subcategory and brand.
- 6. Customer profile features.** Scale age, one-hot encode segment, preferred category, gender and city.
- 7. Customer engagement features.** Interaction counts, spend, conversion rate and first/last seen dates.
- 8. Save everything** as pickle files in data/processed.

4. Key functions

Function	What it does
<code>create_user_item_matrix()</code>	The customer x product rating grid
<code>create_normalized_user_item_matrix()</code>	Same grid, personal average removed
<code>create_implicit_feedback_matrix()</code>	Binary cart/purchase grid
<code>create_item_popularity_features()</code>	Exposure, conversion, long-tail flag
<code>create_item_content_features()</code>	Numeric product features
<code>create_user_profile_features()</code>	Numeric customer features
<code>create_user_engagement_features()</code>	Behavioural summary per customer
<code>mark_relevant_interactions()</code>	Labels used by the evaluation metrics
<code>save_pickle()</code> / <code>load_pickle()</code>	Read and write artifacts

5. Inputs, outputs and how to run

	Detail
Inputs	The three raw tables
Outputs	7 pickle files in data/processed
Key shapes	rating matrix 5,766 x 2,264 • catalogue 2,500 products
Runtime	About 30 seconds
Run with	<code>python src/preprocessing.py</code>

6. Key code

src/preprocessing.py — create_normalized_user_item_matrix()

```

48 | def create_normalized_user_item_matrix(interactions_df: pd.DataFrame) -> pd.DataFrame:
49 |     """Mean-centre each user's ratings so factors capture relative preference."""
50 |     rated = interactions_df.dropna(subset=["rating"]).copy()
51 |
52 |     # transform broadcasts the group mean in one pass; .apply is far slower.
53 |     user_mean = rated.groupby("user_id")["rating"].transform("mean")
54 |     rated["normalized_rating"] = rated["rating"] - user_mean
55 |
56 |     return rated.pivot_table(
57 |         index="user_id",
58 |         columns="item_id",
59 |         values="normalized_rating",
60 |         aggfunc="mean",
61 |         fill_value=0,
62 |     ).astype(np.float32)

```

Note groupby().transform() rather than a row-by-row .apply() - it does the same thing in one pass and is far faster.

7. Things to remember

Why subtract each customer's average rating?

Imagine two customers who like exactly the same products in exactly the same order. One is generous and rates everything 4 or 5; the other is harsh and rates everything 2 or 3. To a model those look like completely different people. Subtracting each person's own average removes that personal style, leaving only *relative* preference - which is what we actually want to learn.

Why the reindex line is not optional

A product with **zero** interactions never appears in the interaction log at all, so a normal groupby silently drops it. Those are exactly the new products the system is supposed to handle. Reindexing against the full catalogue keeps all 2,500 rows. Notice the rating matrix has only 2,264 columns - that gap of 236 products *is* the cold-start problem, made visible.

8. Likely questions

Q. Why is 0 a safe value for an empty cell?

A. Because the rating scale starts at 1. A 0 can only mean 'no rating given' - it can never be confused with a genuine low score.

Q. Why keep both an explicit and an implicit matrix?

A. About 31% of interactions have no star rating, so the explicit matrix simply cannot see them. Cart and purchase events are recorded every time, for everyone, and are not affected by who bothers to leave a review.

Q. Why float32?

A. The rating matrix is 15 million cells. float64 would double the memory for no gain in accuracy at this scale.